FACULTAD DE MATEMÁTICAS - UADY

DESARROLLO DE APLICACIONES WEB

Proyecto Integrador

SLAM SHOES

Prototipo funcional de tienda web para el comercio de calzado deportivo orientado al basquetbol.



Documento del modelo relacional de la base de datos.

Primera revisión - 29 de octubre de 2026

Integrantes del equipo:

CAAMAL TENCLE MATEO ALEJANDRO

DZIB MARTÍNEZ AGUSTIN EMANUEL

ESQUIVEL BACELIS ESTEFANIA

MORALES CARO AXEL ADOLFO

Profesor: Juan Pablo Ucán Pech

Repositorio: https://github.com/estefaniaesquivel/Slam-Shoes-Web

1. Introducción

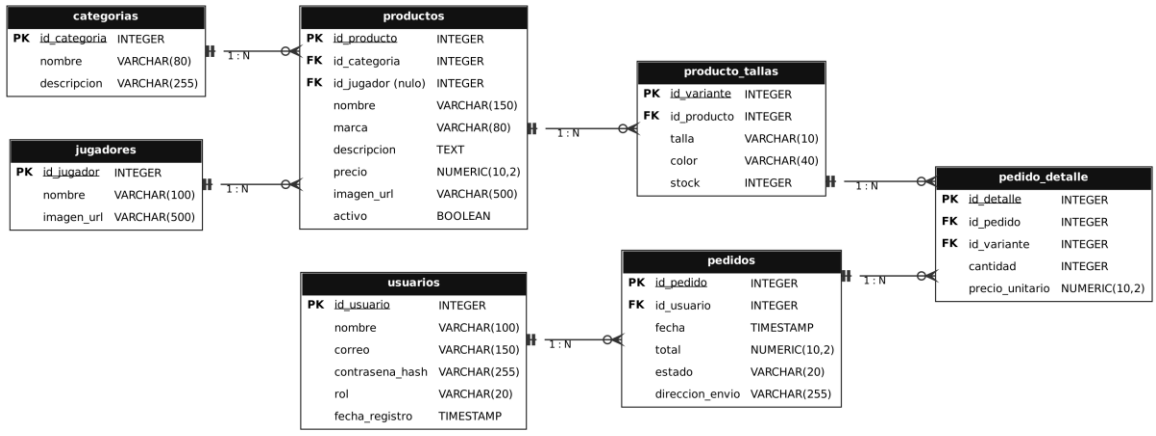
Este documento describe el modelo relacional de la base de datos de Slam Shoes, una tienda web de calzado deportivo orientado al basquetbol. Complementa el documento de requisitos del software y detalla, para cada tabla, los tipos de dato, las llaves y las restricciones. La base de datos es relacional, utiliza PostgreSQL y estará alojada en Supabase. Consta de siete tablas relacionadas entre sí mediante llaves foráneas: usuarios, categorias, jugadores, productos, producto_tallas, pedidos y pedido_detalle.

1.1 Convenciones

- PK: llave primaria. FK: llave foránea.
- Nulo: indica si el campo admite valores nulos (Sí) o es obligatorio (No).
- Los nombres de tablas y campos están en minúsculas, sin acentos y con guion bajo, para evitar problemas en PostgreSQL.
- Las llaves primarias son enteros autogenerados (GENERATED ALWAYS AS IDENTITY).

2. Diagrama entidad-relación

La figura muestra las seis tablas, sus campos principales y las relaciones. La notación es de pata de gallo: la barra doble indica el lado "uno" y la pata de gallo con círculo indica el lado "muchos" (cero o más), así como la notación de esa misma relación, 1:N (one-to-many).



Análisis del diagrama:

Tabla categorias

Campos: id_categoria (PK), nombre, descripcion.

Por qué sus llaves: Contiene únicamente su llave primaria (id_categoria) porque es una tabla padre independiente. No necesita depender de ninguna otra entidad; su propósito es clasificar los productos del inventario.

Tabla jugadores

Campos: id_jugador (PK), nombre, imagen_url.

Por qué sus llaves: Actúa como una tabla catálogo independiente que almacena la información de los atletas profesionales. Su llave primaria (id_jugador) identifica de forma uníca a cada jugador para poder asociarlo con los modelos de calzado que inspiran.

Tabla productos

Campos: id_producto (PK), id_categoria (FK), id_jugador (FK, nulo), nombre, marca, descripcion, precio, imagen_url, activo.

Por qué sus llaves:

- id_producto es la PK que identifica de forma única a cada modelo de tenis.
- id_categoria es FK, ya que crea un puente o enlace directo hacia la tabla de categorías. En lugar de repetir texto, el producto simplemente guarda el número de la categoría para permitir agruparlos y filtrarlos en el catálogo web.
- id_jugador (FK) es una llave foránea opcional (admite valores nulos) que vincula el modelo de tenis con el atleta profesional que lo representa, permitiendo realizar filtros especializados por jugador en la tienda.

Tabla producto_tallas

Campos: id_variante (PK), id_producto (FK), talla, color, stock.

Por qué sus llaves:

- id_variante es la PK que identifica cada combinación física única de talla y color de un modelo.
- id_producto (FK) es indispensable porque un calzado puede tener múltiples variantes (por ejemplo, el modelo viene en talla 26 en color negro, talla 27 en color blanco, etc). La FK vincula cada variante directamente con su producto base en la tabla productos.

Tabla usuarios

Campos: id_usuario (PK), nombre, correo, contrasena_hash, rol, fecha_registro.

Por qué sus llaves: Al igual que categorias, es una tabla principal o independiente. id_usuario actúa como PK para identificar unívocamente a cada cliente o administrador registrado en el sistema.

Tabla pedidos

Campos: id_pedido (PK), id_usuario (FK), fecha, total, estado, direccion_envio.

Por qué sus llaves:

- id_pedido es la PK que funciona como el folio o comprobante general de la compra.
- id_usuario (FK) se coloca obligatoriamente en esta tabla para saber qué usuario realizó el pedido. Permite asociar la compra al historial del cliente y mantener la trazabilidad de quién ordenó los productos.

Tabla pedido_detalle

Campos: id_detalle (PK), id_pedido (FK), id_variante (FK), cantidad, precio_unitario.

Por qué sus llaves:

- id_detalle es la PK que identifica de forma unívoca cada renglón o línea individual dentro del ticket de compra.
- id_pedido (FK) conecta este renglón con su respectivo comprobante general en la tabla pedidos.
- id_variante (FK) conecta el renglón directamente con el stock específico de la tabla producto_tallas, permitiendo especificar la talla y color exactos que el usuario compró, registrar el precio unitario exacto al momento de la transacción y descontar la existencia correcta del inventario.

3. Descripción de las tablas

3.1 usuarios

Almacena a los clientes y administradores. Es la base de la autenticación y de la distinción de roles.

Campo	Tipo de dato	Llave	Nulo	Restricciones / observaciones
id_usuario	INTEGER	PK	No	Autogenerado (IDENTITY).
nombre	VARCHAR(100)		No	Nombre completo del usuario.
correo	VARCHAR(150)		No	UNIQUE. Se usa para iniciar sesión. Le indica a la base de datos que no puede haber dos usuarios registrados con el mismo correo. Si alguien intenta registrarse con un correo que ya existe en la tabla, PostgreSQL rechazará la operación de inmediato.
contrasena_hash	VARCHAR(255)		No	Contraseña cifrada con hash (BCrypt usando en el backend Spring Security).
rol	VARCHAR(20)		No	DEFAULT 'cliente'. Es un valor predeterminado automático. Si al momento de registrar un nuevo usuario desde la API de Spring Boot no se especifica explícitamente un rol, la base de datos le asignará la palabra 'cliente' por defecto. CHECK (rol IN ('cliente','administrador')). Es una restricción (constraint) que actúa como un filtro o policía de seguridad en la base de datos. Le indica a PostgreSQL que únicamente se permiten dos valores válidos para esa columna: 'cliente' o 'administrador'.
fecha_registro	TIMESTAMP		No	DEFAULT CURRENT_TIMESTAMP.

				Si al momento de registrar un usuario desde la API de Spring Boot no se manda una fecha de forma manual, la base de datos rellena este campo por sí sola con la fecha y hora exactas del servidor en el instante preciso en que el usuario se creó.

- Llave primaria: id_usuario.
- Llave candidata única: correo.

3.2 jugadores

Almacena la información de los atletas profesionales que inspiran los modelos de calzado deportivo de la tienda. Permite estructurar el catálogo y realizar filtros especializados en el FrontEnd.

Campo	Tipo de dato	Llave	Nulo	Restricciones / observaciones
id_jugador	INTEGER	PK	No	Autogenerado (IDENTITY).
nombre	VARCHAR(100)		No	Nombre completo del jugador profesional (p. ej. LeBron James, Giannis Antetokounmpo).
imagen_url	VARCHAR(500)		No	Enlace o ruta de la fotografía representativa del jugador para mostrar en la interfaz.

3.3 categorias

Agrupa los productos por tipo de calzado (exterior o interior, su nombre).

Campo	Tipo de dato	Llave	Nulo	Restricciones / observaciones
id_categoria	INTEGER	PK	No	Autogenerado (IDENTITY).
nombre	VARCHAR(80)		No	UNIQUE. No se repiten nombres de categoría.
descripcion	VARCHAR(255)		Sí	Texto descriptivo opcional.

- Llave primaria: id_categoria.

3.4 productos

Catálogo general de modelos de calzado. Cada producto pertenece a una categoría.

Campo	Tipo de dato	Llave	Nulo	Restricciones / observaciones
id_producto	INTEGER	PK	No	Autogenerado (IDENTITY).
id_categoria	INTEGER	FK	No	Referencia a categorias(id_categoria).

Id_jugador	INTEGER	FK	Si	Referencia a jugadores(id_jugador). Opcional (nulo); vincula el modelo al atleta profesional que lo inspira.
nombre	VARCHAR(150)		No	Nombre del modelo.
marca	VARCHAR(80)		No	Marca del calzado.
descripcion	TEXT		Sí	Descripción larga del producto.
precio	NUMERIC(10,2) Se usa NUMERIC en lugar de FLOAT o DOUBLE para el dinero		No	CHECK (precio >= 0). Precio vigente en pesos. Le ordena a PostgreSQL que únicamente acepte precios mayores o iguales a cero.
imagen_url	VARCHAR(500)		Sí	URL de la imagen principal.
activo	BOOLEAN		No	DEFAULT TRUE. FALSE = baja lógica (no se borra el registro).

- Llave primaria: id_producto.
- Llave foránea: id_categoria → categorias.

3.5 producto_tallas

Cada combinación de talla y color de un producto, con sus existencias (variante).

Campo	Tipo de dato	Llave	Nulo	Restricciones / observaciones
id_variante	INTEGER	PK	No	Autogenerado (IDENTITY).
id_producto	INTEGER	FK	No	Referencia a productos(id_producto).
talla	VARCHAR(10)		No	Ej. '26', '27.5'. Se guarda como texto para admitir medias tallas.
color	VARCHAR(40)		No	Color de la variante.
stock	INTEGER		No	DEFAULT 0. CHECK (stock >= 0).

- Llave primaria: id_variante.
- Llave foránea: id_producto → productos.
- Restricción única compuesta: UNIQUE (id_producto, talla, color), para no duplicar una variante.

3.6 pedidos

Encabezado de cada compra realizada por un usuario.

Campo	Tipo de dato	Llave	Nulo	Restricciones / observaciones
id_pedido	INTEGER	PK	No	Autogenerado (IDENTITY).
id_usuario	INTEGER	FK	No	Referencia a usuarios(id_usuario).
fecha	TIMESTAMP		No	DEFAULT CURRENT_TIMESTAMP.
total	NUMERIC(10, 2)		No	CHECK (total >= 0). Suma de los renglones del pedido.

estado	VARCHAR(20)		No	DEFAULT 'pendiente'. CHECK (estado IN ('pendiente','pagado','enviado','entregado','cancelado')).
direccion_envio	VARCHAR(255)		No	Dirección de entrega capturada en el checkout.

- Llave primaria: id_pedido.
- Llave foránea: id_usuario → usuarios.

3.7 pedido_detalle

Renglones de cada pedido: qué variante se compró, en qué cantidad y a qué precio.

Campo	Tipo de dato	Llave	Nulo	Restricciones / observaciones
id_detalle	INTEGER	PK	No	Autogenerado (IDENTITY).
id_pedido	INTEGER	FK	No	Referencia a pedidos(id_pedido).
id_variante	INTEGER	FK	No	Referencia a producto_tallas(id_variante).
cantidad	INTEGER		No	CHECK (cantidad > 0).
precio_unitario	NUMERIC(10,2)		No	CHECK (precio_unitario >= 0). Copia del precio al momento de la compra.

- Llave primaria: id_detalle.
- Llaves foráneas: id_pedido → pedidos; id_variante → producto_tallas.
- Restricción única compuesta: UNIQUE (id_pedido, id_variante), para que una variante aparezca una sola vez por pedido.

4. Relaciones e integridad referencial

Todas las relaciones son de uno a muchos (1:N). La columna ON DELETE indica qué ocurre con los registros hijos cuando se intenta eliminar el registro padre. En cuanto a la regla ON UPDATE, en el diseño de base de datos se configuró de manera estándar con el valor por defecto: NO ACTION (que en la práctica funciona igual que RESTRICT).

Llave foránea (hijo)	Referencia (padre)	Cardinalidad	ON DELETE
productos.id_categoria	categorias.id_categoria	1 a N	RESTRICT
productos.id_jugador	jugadores.id_jugador	1 a N	SET NULL
producto_tallas.id_producto	productos.id_producto	1 a N	RESTRICT
pedidos.id_usuario	usuarios.id_usuario	1 a N	RESTRICT
pedido_detalle.id_pedido	pedidos.id_pedido	1 a N	CASCADE
pedido_detalle.id_variante	producto_tallas.id_variante	1 a N	RESTRICT

Criterios de eliminación

- **RESTRICT** impide borrar un registro que todavía es referenciado. Por ejemplo, no se puede borrar un usuario con pedidos ni una variante que ya fue vendida, de modo que el historial de compras se conserva.

- **SET NULL** en productos.id_jugador permite que, si un jugador profesional es eliminado del catálogo, los modelos de calzado inspirados en él no se borren de la base de datos; en su lugar, la referencia pasa a ser NULL, convirtiendo temporal o permanentemente el calzado en un producto general sin afectar el inventario de la tienda.
- **CASCADE** en pedido_detalle.id_pedido elimina los renglones si se elimina su pedido, porque un renglón no tiene sentido sin su encabezado.
- **Baja lógica:** los productos no se borran físicamente; el administrador los da de baja poniendo activo = FALSE. Así se respeta RF-10 sin romper pedidos antiguos.

5. Restricciones de dominio

Tabla.campo	Regla
usuarios.correo	Valor único; no puede repetirse entre usuarios.
usuarios.rol	Solo admite 'cliente' o 'administrador'.
categorias.nombre	Valor único.
jugadores.nombre	Valor único (UNIQUE). Evita duplicados en el catálogo de atletas profesionales.
productos.precio	Mayor o igual a 0.
producto_tallas.stock	Mayor o igual a 0; no puede quedar existencia negativa.
producto_tallas (id_producto, talla, color)	Combinación única por variante.
pedidos.estado	Solo admite 'pendiente', 'pagado', 'enviado', 'entregado' o 'cancelado'.
pedidos.total	Mayor o igual a 0.
pedido_detalle.cantidad	Mayor que 0.
pedido_detalle (id_pedido, id_variante)	Combinación única; una variante aparece una vez por pedido.

6. Normalización

El modelo cumple con la tercera forma normal (3FN):

- 1FN: Todos los campos son atómicos y no hay grupos repetidos; las tallas y colores de un producto viven en filas separadas de producto_tallas, y los datos de los atletas están aislados en su propia tabla.
- 2FN: Todas las tablas principales utilizan una llave primaria simple (id_...), por lo que no existen dependencias parciales.
- 3FN: Los atributos dependen únicamente de la llave primaria; por ejemplo, la información de la categoría se encuentra exclusivamente en categorias y la del atleta en jugadores, evitando duplicar datos en la tabla de productos.

Hay dos decisiones de diseño que se guardan a propósito para conservar el historial y optimizar consultas: precio_unitario en pedido_detalle (el precio de los productos en el catálogo puede cambiar con el tiempo, pero el histórico de la compra debe preservarse intacto) y total en pedidos (resultado consolidado de sumar cantidad × precio_unitario de sus respectivos renglones).